DATA STRUCTURES LAB – TRACE ANALYSIS WORKBOOK
Linked List Code Tracing and Program Understanding

📄 Introduction

This workbook is designed to strengthen students' understanding of **Linked List operations** through careful **code tracing and logical analysis**. The exercises encourage students to follow the movement of pointers and variables step-by-step and determine the output produced by the given programs.

Students are expected to **analyze each program manually**, track how variables change during execution, and understand the underlying logic behind each operation.

📂 Structure of the Workbook

The workbook is divided into **two main sections**, each focusing on different aspects of linked list programming.

SECTION – 1: Linked List Code Tracing

This section focuses on tracing the behavior of programs written using linked lists. Students must determine the output by analyzing the movement of pointers and variables.

Part A – Singly Linked List (SLL)
Programs related to operations on **Singly Linked Lists**. Students must trace each function and determine the final output.

Part B – Circular Singly Linked List (CSLL)
Programs involving **Circular Singly Linked Lists**, where students analyze pointer movement in circular traversal.

SECTION – 2: Student Application Programs

This section contains application-based programs using a **Student structure**. Students must identify the purpose of the code and determine the result.

Part A – Singly Linked List (SLL) Applications
Programs that manipulate student data using SLL operations.

Part B – Circular Singly Linked List (CSLL) Applications
Programs that perform operations on student data using CSLL.

Instructions for Students

1. Carefully **read and traverse each program step by step**.
2. **Trace the movement of variables and pointers** during execution.
3. Write the **intermediate values** observed during tracing.
4. Determine and write the **final output of the program**.
5. In **Section – 2**, additionally **identify the purpose of the code** (for example: sum calculation, swapping values, updating marks, etc.).
6. Provide both:
    - The **output of the program**
    - The **complete tracing of variable movements**
7. Need to perform this activity in group of 3.

SECTION-1

For SLL

Head → [10] → [20] → [30] → [40] → [50] → [60] → NULL

For CSLL

Head → [10] → [20] → [30] → [40] → [50] → [60] → [70]
^
|_____|

Students must **trace pointer movement step-by-step**.

PART A — SLL

Q1

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p->next!=NULL)
       p=p->next;

   printf("%d",p->data);
}
```

Q2

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p=p->next->next;

   printf("%d",p->data);
}
```

Q3

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p!=NULL)
   {
       printf("%d ",p->data);
       p=p->next->next;
   }
}
```

Q4

```
void traceFunc(struct node *head)
{
   struct node *p=head;
   struct node *q=head->next;

   printf("%d %d",p->data,q->data);
}
```

Q5

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p->next->next!=NULL)
       p=p->next;

   printf("%d",p->data);
}
```

Q6

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   printf("%d ",p->data);

   p=p->next;
   printf("%d ",p->data);

   p=p->next->next;
   printf("%d",p->data);
}
```

Q7

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p!=NULL)
   {
       if(p->data%20==0)
           printf("%d ",p->data);

       p=p->next;
   }
}
```

Q8

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p=p->next;
   p->data=50;
   printf("%d %d",head->data,p->data);
```

```
}
```

Q9

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p->next!=NULL)
   {
       printf("%d ",p->data);
       p=p->next;
   }
}
```

Q10

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   printf("%d ",p->next->data);
   printf("%d ",p->next->next->data);
}
```

Q11

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p->next!=NULL)
   {
       p=p->next;
       printf("%d ",p->data);
   }
}
```

Q12

```
void traceFunc(struct node *head)
{
   struct node *p=head->next->next;

   printf("%d",p->next->data);
}
```

Q13

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p->next=p->next->next;

   while(p!=NULL)
   {
       printf("%d ",p->data);
       p=p->next;
   }
}
```

Q14

```
void traceFunc(struct node *head)
{
   struct node *p=head;
   struct node *q=head;

   q=q->next->next;

   printf("%d %d",p->data,q->data);
}
```

Q15

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   while(p!=NULL)
   {
       printf("%d ",p->data);
       if(p->next!=NULL)
           p=p->next->next;
       else
           p=NULL;
   }
}
```

PART B — CSLL

Q1

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   printf("%d ",p->data);
   p=p->next;

   printf("%d ",p->data);
}
```

Q2

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p=p->next->next;

   printf("%d",p->data);
}
```

Q3

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   do
   {
       printf("%d ",p->data);
       p=p->next;
   }
   while(p!=head);
}
```

Q4

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p=p->next->next->next;

   printf("%d",p->data);
}
```

Q5

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   for(int i=0;i<5;i++)
   {
       printf("%d ",p->data);
       p=p->next;
   }
}
```

Q6

```
void traceFunc(struct node *head)
{
   struct node *p=head->next;

   printf("%d ",p->data);
   printf("%d ",p->next->data);
}
```

Q7

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p=p->next->next->next->next;

   printf("%d",p->data);
}
```

Q8

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   for(int i=0;i<6;i++)
   {
       p=p->next;
   }

   printf("%d",p->data);
}
```

Q9

```
void traceFunc(struct node *head)
{
   struct node *p=head->next->next;

   printf("%d ",p->data);
   printf("%d ",p->next->data);
}
```

Q10

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   do
   {
       printf("%d ",p->data);
       p=p->next->next;
   }
   while(p!=head);
}
```

Q11

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   printf("%d ",p->data);

   p=p->next->next;

   printf("%d ",p->data);
}
```

Q12

```
void traceFunc(struct node *head)
{
   struct node *p=head->next;

   for(int i=0;i<4;i++)
   {
       printf("%d ",p->data);
       p=p->next;
   }
}
```

Q13

```
void traceFunc(struct node *head)
{
   struct node *p=head->next->next;

   printf("%d ",p->data);
   printf("%d ",p->next->next->data);
}
```

Q14

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   p=p->next;

   printf("%d ",p->data);

   p=p->next->next;

   printf("%d",p->data);
}
```

Q15

```
void traceFunc(struct node *head)
{
   struct node *p=head;

   for(int i=0;i<7;i++)
   {
       printf("%d ",p->data);
       p=p->next;
   }
}
```

SECTION-2

Node Structure

```
struct student
{
   int roll;
   int age;
   int marks;
   struct student *next;
};
```

Initial List for SLL

```
Head
 |
 v
[Roll:11 Age:20 Marks:70] → [Roll:14 Age:21 Marks:60] →[Roll:17 Age:22 Marks:80] →[Roll:19 Age:20
Marks:75] → NULL
```

Initial List for CSLL

```
Head
 |
 v
[11,20,70] → [14,21,60] → [17,22,80] → [19,20,75]
 ^                                |              |
 |________________________________________________|
```

Students must **trace pointer movement and predict output**.

PART A — SLL

Q1

```
void traceFunc(struct student *head)
{
   struct student *temp=head;
   int i,flag;

   while(temp!=NULL)
   {
      flag=0;

      for(i=2;i<temp->roll;i++)
      {
         if(temp->roll%i==0)
         {
            flag=1;
            break;
         }
      }

      if(flag==0)
         printf("%d ",temp->roll);

      temp=temp->next;
   }
}
```

Q2

```
void traceFunc(struct student *head)
{
   struct student *temp=head;
   int sum=0;

   while(temp!=NULL)
   {
       sum=sum+temp->roll;
       temp=temp->next->next;
   }

   printf("%d",sum);
}
```

Q3

```
void traceFunc(struct student *head)
{
   struct student *temp=head;
   int sum=0,count=0;

   while(temp!=NULL)
   {
       sum=sum+temp->roll;
       count++;

       if(temp->next!=NULL)
           temp=temp->next->next;
       else
           temp=NULL;
   }

   printf("%d",sum/count);
}
```

Q4

```
void traceFunc(struct student *head)
{
   struct student *temp=head;
   int min=temp->roll;

   while(temp!=NULL)
   {
       if(temp->roll<min)
           min=temp->roll;

       temp=temp->next;
   }

   printf("%d",min);
}
```

Q5

```
void traceFunc(struct student *head)
{
   struct student *temp=head->next;

   while(temp!=NULL)
   {
       printf("%d ",temp->marks);

       if(temp->next!=NULL)
           temp=temp->next->next;
       else
           temp=NULL;
   }
}
```

Q6

```
void traceFunc(struct student *head)
{
   struct student *temp=head;

   while(temp!=NULL)
   {
       if(temp->marks>70)
           printf("%d ",temp->age);

       temp=temp->next;
   }
}
```

Q7

```
void traceFunc(struct student *head)
{
   struct student *temp=head;
   int max=temp->marks;

   while(temp!=NULL)
   {
       if(temp->marks>max)
           max=temp->marks;

       temp=temp->next;
   }

   printf("%d",max);
}
```

Q8

```
void traceFunc(struct student *head)
{
   struct student *temp=head;

   while(temp!=NULL)
   {
       printf("%d ",temp->roll);

       if(temp->next!=NULL)
           temp=temp->next->next;
       else
           temp=NULL;
   }
}
```

Q9

```
void traceFunc(struct student *head)
{
   struct student *temp=head;
   int x;

   x=temp->marks;
   temp->marks=temp->next->marks;
   temp->next->marks=x;

   printf("%d %d",temp->marks,temp->next->marks);
}
```

Q10

```
void traceFunc(struct student *head)
{
   struct student *temp=head;

   if(temp->marks > temp->next->marks)
   {
       int t=temp->marks;
       temp->marks=temp->next->marks;
       temp->next->marks=t;
   }

   printf("%d %d",temp->marks,temp->next->marks);
}
```

PART B — CSLL

Q 1

```
void traceFunc1(struct student *head)
{
    struct student *temp=head;

    temp->marks=temp->marks+10;

    printf("%d",temp->marks);
    temp=temp->next;
}
```

Q 2

```
void traceFunc2(struct student *head)
{
    struct student *temp=head;

    temp->next->marks=temp->next->marks+temp->marks;

    printf("%d",temp->next->marks);
}
```

Q 3

```
void traceFunc3(struct student *head)
{
    struct student *temp=head;

    temp=temp->next->next;

    if(temp->marks>temp->next->marks)
        temp->marks-=5;

    printf("%d",temp->marks);
}
```

Q 4

```
void traceFunc4(struct student *head)
{
    struct student *temp=head;

    int t=temp->marks;
    temp->marks=temp->next->marks;
    temp->next->marks=t;

    printf("%d %d",temp->marks,temp->next->marks);
}
```

Q 5

```
void traceFunc5(struct student *head)
{
    struct student *temp=head;

    temp=temp->next->next;

    temp->next->marks=temp->next->marks+3;

    printf("%d",temp->next->marks);
}
```

Q 6

```
void traceFunc6(struct student *head)
{
    struct student *temp=head;

    temp->marks=temp->marks+temp->next->next->marks;

    printf("%d",temp->marks);
}
```

Q 7

```
void traceFunc7(struct student *head)
{
    struct student *temp=head;

    temp=temp->next;

    if(temp->age==21)
        temp->marks+=7;

    printf("%d",temp->marks);
}
```

Q 8

```
void traceFunc8(struct student *head)
{
    struct student *temp=head;

    temp->next->next->marks-=4;

    printf("%d",temp->next->next->marks);
}
```

Q 9

```
void traceFunc9(struct student *head)
{
    struct student *temp=head;

    int t=temp->next->marks;
    temp->next->marks=temp->next->next->marks;
    temp->next->next->marks=t;

    printf("%d %d",temp->next->marks,temp->next->next->marks);
}
```

Q 10

```
void traceFunc10(struct student *head)
{
    struct student *temp=head;

    temp=temp->next->next->next;

    temp->marks=temp->marks+temp->age;

    printf("%d",temp->marks);
}
```

Q 11

```
void traceFunc11(struct student *head)
{
    struct student *temp=head;

    temp->marks=temp->marks-temp->next->marks;

    printf("%d",temp->marks);
}
```

Q 12

```
void traceFunc12(struct student *head)
{
    struct student *temp=head;

    temp=temp->next;

    temp->marks=temp->marks*2;

    printf("%d",temp->marks);
}
```

Q 13

```
void traceFunc13(struct student *head)
{
    struct student *temp=head;

    temp->next->next->marks=temp->next->next->marks+temp->next->marks;

    printf("%d",temp->next->next->marks);
}
```

Q 14

```
void traceFunc14(struct student *head)
{
    struct student *temp=head;

    temp=temp->next->next;

    int t=temp->marks;
    temp->marks=temp->next->marks;
    temp->next->marks=t;

    printf("%d %d",temp->marks,temp->next->marks);
}
```

Q 15

```
void traceFunc15(struct student *head)
{
    struct student *temp=head;

    temp->marks+=temp->next->next->next->marks;

    printf("%d",temp->marks);
}
```